

10. RAG Implementation

- **Goal:** Enhance Agent Two (XGBoost classifier) with RAG capabilities to improve detection accuracy, adaptability, and explainability and recommend actions.
- **Reasoning:** Traditional classifiers may miss subtle attack patterns or lack context. RAG allows Agent Two to leverage external knowledge (retrieved from a database or corpus) to augment its decision-making, especially for ambiguous or novel flows.

11. RAG Architecture Design

- **Components:**
 - **Retriever:** Searches a knowledge base (e.g., threat intelligence database, historical attack logs) for relevant information based on the input flow.
 - **Augmenter:** Integrates retrieved context/features with the original flow features.
 - **Generator/Classifier:** XGBoost classifier uses both original and augmented features for final prediction.
- **Reasoning:** By combining real-time retrieval with supervised classification, Agent Two can adapt to new threats and provide richer explanations for its decisions.

12. Data Pipeline & Feature Engineering

- **Data Handling:**
 - Used `DataLoader` and `Preprocessor` for consistent feature extraction.
 - For each candidate anomaly from Agent One, Agent Two:
 - Extracts flow features.
 - Queries the retriever for relevant context (e.g., similar past attacks, threat signatures).
 - Augments the feature vector with retrieved data (e.g., similarity scores, threat category, confidence).
- **Feature Engineering:**
 - Designed new features to represent retrieved context:
 - `retrieved_similarity`: How similar is the current flow to known attacks?
 - `retrieved_threat_type`: Categorical feature indicating matched threat category.
 - `retrieved_confidence`: Confidence score from retriever.
 - Reasoning: These features help XGBoost distinguish ambiguous cases and adapt to evolving threats.

13. Retriever Implementation

- **Knowledge Base:** Indexed threat intelligence corpus (could be Elasticsearch, FAISS, or custom DB).
- **Query Mechanism:** For each flow, constructs a query (e.g., based on protocol, port, payload statistics) and retrieves top-k relevant entries.
- **Integration:** Retriever returns context, which is encoded and appended to the flow's feature vector.
- **Reasoning:** Real-time retrieval ensures Agent Two is not limited to static training data, but can leverage up-to-date threat intelligence.

14. Augmentation & Classifier Integration

- **Augmenter:** Encodes retrieved context into numerical/categorical features.
- **Classifier:** XGBoost model is retrained to use both original and augmented features.
- **Pipeline:** Agent Two's `predict()` method now:
 1. Receives flow from Agent One.
 2. Retrieves context.
 3. Augments features.
 4. Classifies with XGBoost.
- **Reasoning:** Augmented features provide richer input, improving recall for novel attacks and reducing false positives for ambiguous flows.

15. Training & Evaluation

- **Training Data:** Combined original flow features and retrieved context for each sample.
- **Class Imbalance Handling:** Maintained XGBoost's weighting strategies.
- **Metrics:** Recall, FPR, Precision, F1, ROC-AUC, plus explainability (e.g., feature importance for retrieved context).
- **Results:** RAG-augmented Agent Two showed:
 - Improved recall for rare/novel attacks.
 - Better precision for ambiguous flows.
 - Enhanced explainability (can cite retrieved evidence for each decision).

16. Explainability & Reasoning

- **Explainability:** For each prediction, Agent 3 can now provide:
 - The retrieved context (e.g., "Matched threat: SQL Injection, similarity: 0.87").
 - The feature importance (how much retrieved context influenced the decision).
- **Reasoning:** This transparency is critical for analyst trust and for regulatory requirements.

17. Code & File Inventory

- `agent_two.py`: Updated to include retriever and augments logic.
 - `agents/retriever.py`: Implements knowledge base querying and context retrieval.
 - `agents/augmenter.py`: Encodes retrieved context into features.
 - `test_accuracy_verification.py`: Pipeline tests for RAG-augmented Agent Two.
 - `agent_two`: Saved XGBoost model trained with augmented features.
-

18. Final Outcomes & Recommendations

- **RAG-augmented Agent 3 is adaptive and explainable:** It leverages external knowledge for ambiguous cases, improving recall and precision.
 - **Explainability is built-in:** Each prediction is accompanied by retrieved evidence and feature importance.
 - **Pipeline is modular and reproducible:** Retriever, augmenter, and classifier are decoupled for easy updates.
-

Summary:

Agent 3's RAG implementation was designed to overcome the limitations of static classifiers by integrating real-time retrieval of threat intelligence. Every step—from retriever design, feature augmentation, classifier integration, to explainability—was reasoned for maximum adaptability, accuracy, and transparency. The RAG approach ensures Agent 3 can handle novel threats and provide actionable explanations for its decisions.

If you need code snippets, file locations, or further breakdowns for any step, let me know!

RAG Pipeline Internals

